UniApply SA

Complete Project Documentation, Architecture and Starter Foundation

A structured specification for a South African university application management platform

Current stack decision: HTML, CSS, Vanilla JavaScript, Supabase Auth, Express, Render PostgreSQL, Cloudinary, Vercel, Render and GitHub

DOCUMENT STATUS

Contents

Note: This document records the decisions made in the UniApply SA discussion. It describes both the future system and the much smaller first task for Codex: create the structured project foundation without implementing the full MVP.

1. Document Purpose and Current Decisions

This document consolidates the complete UniApply SA idea, expected user experience, architectural approach, design patterns, database direction, deployment model, repository organisation, individual file responsibilities and recommended development sequence.

The immediate objective is not to build the full platform. The first Codex task is to create a clean, scalable and runnable repository foundation so that the team can implement features one phase at a time.

1.1 Confirmed technology decisions

Area	Confirmed decision
Frontend	Semantic HTML5, plain CSS3 and vanilla JavaScript using browser ES modules
Authentication	Supabase Auth
Backend	Node.js and Express.js REST API
Application database	Render PostgreSQL
Initial file storage	Cloudinary
Future file storage	AWS S3 through a storage adapter
Frontend hosting	Vercel
Backend hosting	Render Web Service
Source control	GitHub
Testing	Jest and Supertest
Automation later	Playwright or another JavaScript browser-automation tool, only where permitted

1.2 Supabase and Render responsibilities

Note: Supabase is used for authentication only. Render PostgreSQL stores the application data. This avoids maintaining two competing application databases.

- **Supabase Auth:** registration, login, logout, password recovery, sessions and access tokens.
- **Render PostgreSQL:** student profiles, results, documents metadata, universities, programmes, carts, quotes, payments, applications, status histories, notifications and audit records.
- **Connection between systems:** each local application user record stores the authenticated Supabase user identifier in a unique supabase_user_id field.
- **Passwords:** must never be copied into Render PostgreSQL. Password handling remains the responsibility of Supabase Auth.

1.3 Current implementation boundary

- Create all agreed folders and starter files.
- Provide a basic Express server and /api/health endpoint.
- Prepare Supabase, PostgreSQL, Cloudinary, Vercel and Render configuration.
- Create semantic HTML boilerplates and minimal shared CSS.
- Create valid placeholder exports, controllers, services, repositories, adapters and middleware.
- Do not build authentication screens, profile forms, APS logic, recommendations, payments, application automation or the final database schema yet.

2. Project Overview and Value Proposition

2.1 Project name

UniApply SA — South African University Application Management Platform

2.2 Main idea

UniApply SA is intended to help matric students enter their information once, select South African universities and programmes, understand their eligibility, pay the required charges and track each university application from preparation to final outcome.

2.3 Problem being solved

- Students currently repeat the same personal and academic information on multiple university websites.
- Different universities calculate APS and apply programme requirements differently.
- Students can miss documents, subject requirements, deadlines and application updates.
- Application fees and service charges can be difficult to understand across multiple institutions.
- Tracking several applications across separate portals and email accounts is inconvenient.
- Administrators need a controlled queue and clear hand-off points when automation cannot proceed.

2.4 Proposed value

- One reusable student profile.
- University-specific APS calculations and programme requirement checks.
- Transparent programme recommendations with reasons.
- A single selection cart and fee quotation.
- Application status tracking with a permanent history.
- Notifications for students and administrators.
- A manual-admin workflow first, followed by carefully controlled automation later.

3. Users, Roles and Responsibilities

Role	Primary responsibilities
Student	Register, complete profile, add marks, upload documents, browse programmes, receive recommendations, select choices, pay, track applications and read notifications.
Application Officer / Admin	Review applications, view relevant student data, assign work, update statuses, request missing information and record notes.
Content Administrator	Maintain universities, programmes, APS methods, fees, dates and source verification.
Super Administrator	Manage staff roles, system configuration, permissions, integrations and audit review.
Support Officer	Help students resolve account, profile, payment and application issues without receiving unnecessary privileges.

Note: The initial technical foundation prepares only STUDENT and ADMIN constants. More specialised roles can be added later using role-based access control.

4. Functional Scope and Workflows

4.1 Student journey

```
Landing page
    ↓
Register or log in with Supabase Auth
    ↓
Complete onboarding and student profile
    ↓
Add Grade 11 / Grade 12 results
    ↓
Upload supporting documents
    ↓
Browse universities and programmes
    ↓
Calculate university-specific APS
    ↓
View eligibility and recommendations
    ↓
Select universities and programme choices
    ↓
Resolve missing requirements
    ↓
Receive fee quotation
    ↓
Complete payment
    ↓
Applications enter admin processing queue
    ↓
Track statuses and receive notifications
```

4.2 Student features

- Landing, about, registration and login pages.
- Profile onboarding with completion indicators.
- Personal, contact, address, guardian and school information.
- Academic-result entry for Grade 11 final, Grade 12 June, trial and final results.
- Document upload for ID/passport, academic reports and proof of address.
- University and programme catalogue.
- University-specific APS calculation.
- Rule-based programme recommendations.
- Persistent programme selection cart.
- Requirements checking with links to incomplete sections.
- Fee quotation and later payment integration.
- Application tracking, timeline and notifications.

4.3 Admin features

- Dashboard statistics.
- A prioritised or sampled list of approximately 5–10 applications, with 8 as the initial target.
- View-all-applications page with search, filtering, sorting and pagination.
- Student profile, results, document metadata, programme choices and payment details.
- Assignment of applications to administrators.
- Controlled status changes using a state machine.
- Action-required messages visible to the student.
- Internal notes and audit records.
- Management of universities, programmes and admission rules in later phases.

4.4 Profile completeness and requirements engine

A student should not be blocked simply because the entire profile is not 100% complete. The system must check the specific requirements of the selected university and programme.

```
Selected programme requires:
✓ Personal information
✓ Mathematics result
✓ English result
✗ Grade 12 June report
✗ Proof of address

Result: checkout is blocked and the student receives direct links to the Documents section.
```

A missing-requirement response should include the affected section, field, explanatory message and target page. The backend must enforce this rule; frontend validation alone is insufficient.

4.5 APS and recommendations

- APS calculations differ by university and therefore use the Strategy pattern.
- The initial recommendation system is rule-based, not machine learning.
- It compares APS, required subjects, subject minimums and missing information.
- Recommended result categories are STRONG_MATCH, ELIGIBLE, BORDERLINE, NOT_ELIGIBLE and INSUFFICIENT_INFORMATION.
- Every result must include a human-readable explanation.
- Requirements must be versioned by academic year and verified against official university sources before production use.
- Meeting minimum requirements must never be presented as a guarantee of admission.

4.6 Cart, quotation and payment

Students can select several universities and order multiple programme choices per university. Duplicate programme selections must be prevented.

```
Total payable = sum of verified university/application-channel fees
              + R10 service fee per unique university
              + any clearly disclosed payment-provider fee
```

The backend calculates the final amount and stores a quotation snapshot. The first implemented payment flow may be a clearly labelled demonstration provider; real card information must not be collected directly by UniApply SA.

4.7 Application tracking and state control

Every application has a current status and a permanent status-history record. Invalid status changes must be rejected.

```
DRAFT → PROFILE_INCOMPLETE or READY_FOR_PAYMENT
READY_FOR_PAYMENT → PAYMENT_PENDING
PAYMENT_PENDING → PAID
PAID → QUEUED
QUEUED → IN_PROGRESS
IN_PROGRESS → ACTION_REQUIRED, SUBMITTED or FAILED
SUBMITTED → ACKNOWLEDGED or UNDER_REVIEW
UNDER_REVIEW → WAITLISTED, OFFERED or REJECTED
```

4.8 Notifications and future email aliases

- In-app notifications for profile, quote, payment, queue, submission, action required, offer and rejection events.
- Unread counts and mark-as-read actions.
- Future application-specific email aliases to match university messages to the correct application.
- Future extraction of deadlines, references and actions from inbound university emails.
- Low-confidence email extraction must be reviewed by an administrator, and the original message must remain available.

4.9 Future application automation

Automation is a later feature and must act as an assistant to an administrator, not as an uncontrolled bot. It may fill supported fields, upload approved documents and stop at unsupported or legally sensitive steps.

- Do not bypass CAPTCHA or multifactor authentication.
- Do not accept declarations on behalf of students without a valid process and consent.
- Do not falsify information or submit unapproved data.
- Do not store university passwords in plain text.
- Use a university adapter per supported application channel.
- Create an ACTION_REQUIRED admin task when automation cannot continue.

5. Architectural Structure

5.1 Multi-tier architecture

```
Vercel static frontend
HTML + CSS + browser JavaScript
            ↓
Supabase Auth
Registration, login, sessions and access tokens
            ↓
Render REST API
Express routes and controllers
            ↓
Service layer
Business and workflow rules
            ↓
Repository layer
Parameterized PostgreSQL queries
            ↓
Render PostgreSQL
Application data
```

5.2 File-upload flow

```
Browser document form
       ↓
Render document endpoint
       ↓
Supabase authentication middleware
       ↓
Multer validation
       ↓
Document service
       ↓
StorageAdapterFactory
       ↓
CloudinaryStorageAdapter initially
       ↓
Cloudinary file storage
       ↓
Document metadata saved in Render PostgreSQL
```

5.3 Why use a modular monolith first

The first backend should be one well-organised Express application divided into feature modules and layers. This is easier to deploy, debug and test than multiple microservices. Background workers and separate automation services can be introduced only when the workload requires them.

5.4 Responsibilities of each layer

Layer	Responsibility
Presentation	Semantic HTML, CSS, browser behaviour, forms, rendering and calls to the REST API.
Authentication	Supabase user accounts, sessions and access tokens.
Routes	Map HTTP methods and paths to controllers; apply middleware.
Controllers	Read request data, call services and return consistent JSON responses.
Services	Profile rules, APS, eligibility, requirements, fees, applications, notifications and workflow coordination.
Repositories	All PostgreSQL queries and persistence operations.
Adapters	Cloudinary/S3, payment, email and future university integrations.
Database	Render PostgreSQL tables, constraints, indexes and transactions.

6. Design Patterns

Pattern	How it helps UniApply SA
MVC	HTML pages are views, Express controllers handle requests, and repositories/domain records represent models.
Service Layer	Keeps business rules out of routes and controllers.
Repository	Keeps SQL in one data-access layer and improves testability.
Strategy	Supports different APS and recommendation algorithms.
Factory	Selects APS, storage, notification, payment or university adapters.
Adapter	Hides provider-specific details such as Cloudinary, S3 or university systems.
Observer / Events	Responds to application changes by recording history and creating notifications.
State / State Machine	Controls valid application-status transitions.
Specification	Combines eligibility and application requirements into reusable rules.
Facade	Coordinates complex operations such as checkout and application creation behind one service.
Template Method	Provides a common future university-application sequence with university-specific steps.
Middleware	Authentication, role checks, validation, uploads, rate limiting, logging and error handling.

7. Technology and Deployment Stack

Component	Technology	Purpose
Frontend	HTML, CSS, Vanilla JavaScript	Static user interface and browser logic.
Auth client	@supabase/supabase-js	Registration, login, logout and session access.
Backend	Node.js + Express	REST API and business logic.
Database client	pg	Render PostgreSQL connection pool and parameterized queries.
File handling	Multer	Incoming upload validation before storage.
Initial storage	Cloudinary	Student documents and media.
Future storage	AWS S3	Scalable object storage through the same adapter interface.
Security	Helmet, CORS, rate limiting	Basic API hardening and request controls.
Testing	Jest + Supertest	Unit and API integration tests.
Frontend hosting	Vercel	Static client deployment from GitHub.
Backend hosting	Render	Node/Express web service.
Database hosting	Render PostgreSQL	Managed application database.
Source control	GitHub	Branches, pull requests and deployment source.

7.1 Deployment topology

```
GitHub repository
   ├── client/  → Vercel static deployment
   └── server/  → Render Web Service

Supabase Auth       → authentication identities and sessions
Render PostgreSQL   → application data
Cloudinary          → initial uploaded-file storage
AWS S3              → later storage replacement
```

7.2 Frontend authentication request flow

```
Student logs in with Supabase
        ↓
Supabase returns a session and access token
        ↓
apiClient.js adds Authorization: Bearer <token>
        ↓
Render API verifies token using Supabase
        ↓
API loads local user role from Render PostgreSQL
        ↓
Protected service executes
```

7.3 Vercel configuration

- Project root directory: client.
- Framework preset: Other.
- No frontend framework build is required for the initial static site.
- Public Supabase URL and anonymous key may be present in frontend configuration; server secrets may not.
- The API base URL points to the Render backend in production.

7.4 Render configuration

- Deploy the Express API as a Render Web Service.
- Use /api/health as the health-check path.
- Store DATABASE_URL, Supabase server credentials and Cloudinary secrets in Render environment settings.
- Create Render PostgreSQL separately and attach its connection URL to the backend.
- Do not embed real secrets in render.yaml or GitHub.

8. Data and Database Design

8.1 Core principles

- Render PostgreSQL is the system of record for application data.
- Supabase stores authentication identities; Render stores a unique supabase_user_id reference.
- Passwords are never stored in application tables.
- Uploaded files are stored in Cloudinary/S3; PostgreSQL stores metadata and provider identifiers.
- All SQL belongs in repository files or migration/seed scripts.
- Use foreign keys, unique constraints, indexes, timestamps and transactions.
- Do not use comma-separated lists for relationships.
- Version APS rules, fees and programme requirements by academic year.

8.2 Proposed tables

Domain	Tables
Identity and access	users, roles, user_roles

Domain	Tables
Student profile	student_profiles, student_addresses, guardians, schools, student_school_records
Academic information	subjects, student_results
Documents	document_types, student_documents
University catalogue	universities, university_assets, campuses, faculties, programmes, intakes
Admission rules	admission_rule_sets, aps_score_bands, admission_requirements
Recommendations	eligibility_evaluations, recommendation_runs, recommendation_items
Rankings and careers	ranking_lists, ranking_items
Selections and fees	application_carts, cart_items, application_channels, fee_schedules, quotes, quote_items
Payments	payments
Applications	applications, application_choices, application_requirements, application_status_history
Admin processing	admin_tasks, admin_notes, automation_jobs, automation_steps
Communication	application_email_aliases, inbound_emails, extracted_email_information, notifications
Compliance and audit	consent_records, audit_logs, data_access_logs

8.3 Important fields and relationships

- **users.supabase_user_id:** unique UUID linking a Supabase identity to a local application user.
- **student_documents:** stores student_id, document type, provider identifier, secure path, file type, verification status and timestamps.
- **admission_rule_sets:** links university/programme rules to an academic year and official source information.
- **applications:** one record per selected university/application channel, with several ordered application choices.
- **application_status_history:** append-only record of every status change.
- **quotes and quote_items:** immutable pricing snapshot used for payment and auditing.
- **audit_logs:** records sensitive administrator actions and application status updates.

8.4 Foundation phase database scope

Note: The first Codex task creates database/schema.sql, database/seed.sql and empty migration/seed folders with explanatory comments only. The full schema is implemented in later phases.

9. REST API Design

9.1 Response format

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}

{
  "success": false,
  "message": "Required information is missing.",
  "code": "PROFILE_INCOMPLETE",
  "errors": []
}
```

9.2 Planned endpoint groups

Group	Planned routes
Health	GET /api/health
Profile	GET/PATCH /api/students/me/profile and profile sections
Results	GET/POST/PATCH/DELETE /api/students/me/results
Documents	GET/POST/DELETE and protected download endpoints
Universities	GET /api/universities and /api/universities/:id

Group	Planned routes
Programmes	GET programme details and requirements
APS and recommendations	GET APS and POST recommendation evaluation
Cart	GET/POST/PATCH/DELETE items and validate cart
Quotes and payments	POST quote, retrieve quote and later payment endpoints
Applications	Student application list, details and timeline
Notifications	List, unread count, read and read-all actions
Admin	Dashboard, applications, assignment, status updates, notes and students

9.3 Foundation phase API scope

- Implement GET /api/health with a success response.
- Feature routes may be registered as 501 Not Implemented placeholders.
- Do not add real business logic or database queries yet.
- Use centralised 404 and error responses.

10. Repository Structure

```
uniapply-sa/
├── client/
│   ├── index.html
│   ├── pages/
│   │   ├── public/
│   │   ├── student/
│   │   └── admin/
│   ├── css/
│   ├── js/
│   │   ├── auth/
│   │   ├── api/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── guards/
│   │   ├── config/
│   │   └── utils/
│   ├── assets/
│   └── vercel.json
├── server/
│   ├── src/
│   │   ├── config/
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── strategies/
│   │   ├── adapters/
│   │   ├── factories/
│   │   ├── events/
│   │   ├── middleware/
│   │   ├── validators/
│   │   ├── constants/
│   │   ├── utils/
│   │   ├── app.js
│   │   └── server.js
│   ├── tests/
│   └── package.json
├── database/
│   ├── migrations/
│   ├── seeds/
│   ├── schema.sql
│   └── seed.sql
├── docs/
├── .github/workflows/ci.yml (optional)
├── render.yaml
├── .env.example
├── .gitignore
├── AGENTS.md
├── package.json
└── README.md
```

11. File-by-File Responsibilities

This section explains how every major starter file contributes to the architecture. Feature files begin as valid placeholders and are filled in during their implementation phase.

11.1 Root files

`package.json` — Defines project metadata, dependencies and commands such as development, start and test scripts.

`.env.example` — Lists required environment variables without real secrets and acts as the template for local and hosted configuration.

`.gitignore` — Prevents environment secrets, node_modules, logs, coverage reports and generated deployment files from entering Git.

`AGENTS.md` — Provides permanent coding rules for Codex and future contributors so generated code preserves the agreed architecture.

`README.md` — Introduces the project, stack, architecture, setup instructions, deployment process, current status and next development phase.

`render.yaml` — Describes the Render Web Service build, start and health-check configuration without embedding secrets.

`.github/workflows/ci.yml` — Optional continuous-integration workflow for dependency installation, syntax checks and tests. Vercel and Render still deploy directly from GitHub.

11.2 Public and student HTML pages

`client/index.html` — Landing page with the UniApply SA introduction and public navigation.

`client/pages/public/about.html` — Explains the platform, process, limitations and important notices.

`client/pages/public/login.html` — Supabase login form and role-aware redirect entry point.

`client/pages/public/register.html` — Supabase student registration form and onboarding redirect entry point.

`client/pages/student/dashboard.html` — Student summary for profile progress, recommendations, applications and notifications.

`client/pages/student/profile.html` — Personal, contact, address, guardian and school information forms.

`client/pages/student/results.html` — Academic subject and percentage management.

`client/pages/student/documents.html` — Secure supporting-document upload and document-status interface.

`client/pages/student/universities.html` — Searchable and filterable university catalogue.

`client/pages/student/university-details.html` — Full details, dates, fees, campuses, APS information and programmes for one university.

`client/pages/student/programmes.html` — Programme catalogue, filters, eligibility display and add-to-cart actions.

`client/pages/student/recommendations.html` — Explained APS and programme recommendation results.

`client/pages/student/cart.html` — Persistent selected universities and ordered programme choices.

`client/pages/student/checkout.html` — Requirements review, quotation and payment entry point.

`client/pages/student/applications.html` — List of all applications belonging to the logged-in student.

`client/pages/student/application-details.html` — One application, its choices, actions and complete status timeline.

`client/pages/student/notifications.html` — Unread/read notification list and notification actions.

11.3 Admin HTML pages

`client/pages/admin/dashboard.html` — Statistics and a manageable set of applications for quick review.

`client/pages/admin/applications.html` — Complete searchable, filterable and paginated application table.

`client/pages/admin/application-details.html` — Detailed processing screen for assignment, statuses, notes and action requests.

`client/pages/admin/students.html` — Student search and basic record access subject to backend authorisation.

`client/pages/admin/universities.html` — Future university catalogue administration.

`client/pages/admin/programmes.html` — Future programme and programme-status administration.

`client/pages/admin/admission-rules.html` — Future APS, subject, document and academic-year rule management.

`client/pages/admin/notifications.html` — Administrator alerts for new, failed, urgent or action-required work.

11.4 CSS files

`client/css/variables.css` — Central CSS custom properties for colours, spacing, radiuses, shadows and content width.

`client/css/reset.css` — Removes inconsistent browser defaults and establishes predictable box sizing and element behaviour.

`client/css/global.css` — Shared body, typography, links, form and general page styles.

`client/css/layouts.css` — Header, navigation, sidebar, dashboard, grid and page-container layouts.

`client/css/components.css` — Reusable button, card, table, modal, toast, badge, form and progress styles.

`client/css/public.css` — Landing, about, login and registration page styles.

`client/css/student.css` — Student-dashboard, profile, recommendation, cart and timeline-specific styles.

`client/css/admin.css` — Admin dashboard, table, processing panel and sidebar-specific styles.

`client/css/responsive.css` — Media queries and mobile/tablet adaptations.

11.5 Frontend authentication and configuration

`client/js/auth/supabaseClient.js` — Creates one reusable browser Supabase client using the public URL and anonymous key.

`client/js/auth/authService.js` — Wraps register, login, logout and current-user functions so pages do not call Supabase directly.

`client/js/auth/sessionManager.js` — Retrieves access tokens, observes session changes and supports redirect behaviour.

`client/js/config/appConfig.js` — Holds public frontend configuration such as API_BASE_URL, SUPABASE_URL and SUPABASE_ANON_KEY.

`client/js/config/environment.example.js` — Documents the public values needed to create the real frontend configuration file.

11.6 Frontend API modules

`client/js/api/apiClient.js` — Central Fetch wrapper that obtains the Supabase token, adds the Bearer header, parses JSON and handles errors.

`client/js/api/profileApi.js` — Student-profile REST requests.

`client/js/api/resultsApi.js` — Academic-result create, read, update and delete requests.

`client/js/api/documentsApi.js` — Document list, upload, download and delete requests; never calls Cloudinary directly.

`client/js/api/universityApi.js` — University-list and university-details requests.

`client/js/api/programmeApi.js` — Programme-list, detail and requirements requests.

`client/js/api/recommendationApi.js` — APS and recommendation requests.

`client/js/api/cartApi.js` — Cart retrieval, add, remove, reorder and validation requests.

`client/js/api/applicationApi.js` — Student application list, detail and timeline requests.

`client/js/api/paymentApi.js` — Quote creation, quote retrieval and later provider-payment requests.

`client/js/api/notificationApi.js` — Notification list, unread count and read-state requests.

11.7 Frontend reusable components

`client/js/components/navbar.js` — Shared navigation, session display, logout and unread-notification indicator.

`client/js/components/sidebar.js` — Student/admin sidebar rendering, active state and mobile collapse behaviour.

`client/js/components/modal.js` — Reusable confirmation and information modal behaviour.

`client/js/components/toast.js` — Consistent temporary success, warning and error messages.

`client/js/components/statusBadge.js` — Consistent visual and textual application-status labels.

`client/js/components/universityCard.js` — Reusable rendering for one university catalogue item.

`client/js/components/programmeCard.js` — Reusable programme, eligibility and add-to-cart card.

`client/js/components/applicationTimeline.js` — Reusable status-history timeline for student and admin views.

11.8 Frontend page modules

`client/js/pages/landing.js` — Landing-page interactivity such as mobile navigation and future sliders or FAQs.

`client/js/pages/login.js` — Login form validation, Supabase login and role/profile redirect logic.

`client/js/pages/register.js` — Registration form validation and Supabase sign-up flow.

`client/js/pages/studentDashboard.js` — Loads student dashboard data and renders summary cards.

`client/js/pages/profile.js` — Loads, validates and saves profile sections.

`client/js/pages/results.js` — Displays and manages academic-result records.

`client/js/pages/documents.js` — Validates files, creates FormData and displays upload results.

`client/js/pages/universities.js` — Loads, searches, filters and renders university cards.

`client/js/pages/universityDetails.js` — Reads the selected university identifier and loads its details.

`client/js/pages/programmes.js` — Loads, filters and selects programmes.

`client/js/pages/recommendations.js` — Renders APS values, match categories and recommendation reasons.

`client/js/pages/cart.js` — Renders selected programmes, ordering, removal and validation.

`client/js/pages/checkout.js` — Loads requirements and quotation and initiates the selected payment flow.

`client/js/pages/applications.js` — Renders the logged-in student's application list.

`client/js/pages/applicationDetails.js` — Renders one application and its status timeline.

`client/js/pages/notifications.js` — Loads notifications and handles read actions.

`client/js/pages/adminDashboard.js` — Loads statistics and selected application cards.

`client/js/pages/adminApplications.js` — Controls admin search, filters, sorting and pagination.

`client/js/pages/adminApplicationDetails.js` — Controls assignment, status changes, notes and action-required messages.

11.9 Frontend guards and utilities

`client/js/guards/authGuard.js` — Redirects unauthenticated visitors away from protected pages.

`client/js/guards/studentGuard.js` — Checks the local role before displaying student-only pages; backend enforcement remains mandatory.

`client/js/guards/adminGuard.js` — Checks the local role before displaying admin pages; backend enforcement remains mandatory.

`client/js/utils/validators.js` — Reusable browser validation such as email, phone and percentage checks.

`client/js/utils/formatters.js` — Currency, date, status and percentage formatting.

`client/js/utils/constants.js` — Shared province, result type, document type and status values.

`client/js/utils/dom.js` — Small DOM helpers for selecting elements, rendering, clearing and loading states.

11.10 Frontend assets and Vercel

`client/assets/images/` — General images, hero graphics and temporary student illustrations.

`client/assets/icons/` — Navigation and interface icons.

`client/assets/logos/` — UniApply and local placeholder university logos.

`client/vercel.json` — Static hosting, clean route, rewrite, header and cache configuration for Vercel.

11.11 Backend configuration

`server/src/config/database.js` — Creates the reusable pg Pool from DATABASE_URL and enables production SSL where required.

`server/src/config/supabase.js` — Creates the trusted server-side Supabase client for token verification and authorised auth operations.

`server/src/config/cloudinary.js` — Configures Cloudinary using server-only credentials.

`server/src/config/environment.js` — Loads and validates required environment variables and gives clear startup errors.

`server/src/config/logger.js` — Central logging without exposing tokens or sensitive student information.

11.12 Backend routes

`server/src/routes/healthRoutes.js` — Defines GET /api/health for local checks and Render health monitoring.

`server/src/routes/profileRoutes.js` — Maps profile endpoints to profile controllers and protection middleware.

`server/src/routes/resultRoutes.js` — Maps academic-result endpoints.

`server/src/routes/documentRoutes.js` — Maps protected document upload, metadata, download and delete endpoints.

`server/src/routes/universityRoutes.js` — Maps public or authenticated university catalogue endpoints.

`server/src/routes/programmeRoutes.js` — Maps programme and requirements endpoints.

`server/src/routes/recommendationRoutes.js` — Maps APS and recommendation endpoints.

`server/src/routes/cartRoutes.js` — Maps selection-cart endpoints.

`server/src/routes/applicationRoutes.js` — Maps student application list, detail and timeline endpoints.

`server/src/routes/paymentRoutes.js` — Maps quotation and later payment/provider-webhook endpoints.

`server/src/routes/notificationRoutes.js` — Maps notification endpoints.

`server/src/routes/adminRoutes.js` — Maps admin dashboard, application, student and management endpoints with role protection.

11.13 Backend controllers

`server/src/controllers/profileController.js` — Receives profile HTTP requests, calls profileService and returns JSON.

`server/src/controllers/resultController.js` — Receives academic-result requests.

`server/src/controllers/documentController.js` — Receives document requests and passes validated uploads to documentService.

`server/src/controllers/universityController.js` — Receives university catalogue requests.

`server/src/controllers/programmeController.js` — Receives programme and requirements requests.

`server/src/controllers/recommendationController.js` — Receives APS and recommendation requests.

`server/src/controllers/cartController.js` — Receives cart requests.

`server/src/controllers/applicationController.js` — Receives student application requests.

`server/src/controllers/paymentController.js` — Receives quotation and payment requests.

`server/src/controllers/notificationController.js` — Receives notification requests.

`server/src/controllers/adminController.js` — Receives admin dashboard and processing requests.

11.14 Backend services

`server/src/services/userService.js` — Synchronises a verified Supabase identity with the local users table and resolves the application role.

`server/src/services/profileService.js` — Profile rules, ownership, updates and completion calculation.

`server/src/services/resultService.js` — Academic-result validation and selection of the latest usable result set.

`server/src/services/documentService.js` — File rules, storage adapter coordination, metadata persistence and ownership checks.

`server/src/services/universityService.js` — University availability, dates and catalogue rules.

`server/src/services/programmeService.js` — Programme information and admission requirements.

`server/src/services/apsService.js` — Chooses and executes the appropriate APS strategy.

`server/src/services/eligibilityService.js` — Checks APS, required subjects and subject minimums.

`server/src/services/recommendationService.js` — Ranks or filters programmes and returns explained results.

`server/src/services/requirementService.js` — Checks profile, result and document requirements before checkout.

`server/src/services/cartService.js` — Prevents duplicates and enforces valid programme-choice rules.

`server/src/services/feeService.js` — Calculates verified application fees and R10 per unique university.

`server/src/services/applicationService.js` — Creates applications, applies state transitions, records history and emits events.

`server/src/services/paymentService.js` — Creates quotation/payment records and coordinates the active payment adapter.

`server/src/services/notificationService.js` — Creates, lists and marks notifications through NotificationFactory and repository calls.

11.15 Backend repositories

`server/src/repositories/userRepository.js` — Local user queries by supabase_user_id, email and role.

`server/src/repositories/profileRepository.js` — Student profile, address, guardian and school queries.

`server/src/repositories/resultRepository.js` — Academic-result queries.

`server/src/repositories/documentRepository.js` — Document metadata and ownership queries.

`server/src/repositories/universityRepository.js` — University catalogue queries.

`server/src/repositories/programmeRepository.js` — Programme and admission-rule queries.

`server/src/repositories/cartRepository.js` — Cart, cart item and choice-order queries.

`server/src/repositories/applicationRepository.js` — Application, choices and status-history queries.

`server/src/repositories/paymentRepository.js` — Quote, quote item and payment queries.

`server/src/repositories/notificationRepository.js` — Notification list, count and read-state queries.

11.16 Strategies and factories

`server/src/strategies/aps/BaseApsStrategy.js` — Defines the expected APS strategy methods.

`server/src/strategies/aps/WitsApsStrategy.js` — Future verified Wits APS implementation.

`server/src/strategies/aps/UjApsStrategy.js` — Future verified UJ APS implementation.

`server/src/strategies/aps/UpApsStrategy.js` — Future verified UP admission-score implementation.

`server/src/strategies/recommendations/BaseRecommendationStrategy.js` — Defines a common recommendation interface.

`server/src/strategies/recommendations/EligibilityRecommendationStrategy.js` — Initial transparent rule-based recommendation approach.

`server/src/factories/ApsStrategyFactory.js` — Returns the correct APS strategy from a university key.

`server/src/factories/StorageAdapterFactory.js` — Returns Cloudinary or S3 according to STORAGE_PROVIDER.

`server/src/factories/NotificationFactory.js` — Creates consistent notification titles and messages.

`server/src/factories/UniversityAdapterFactory.js` — Returns the correct future university/application-channel adapter.

11.17 Adapters

`server/src/adapters/storage/BaseStorageAdapter.js` — Defines upload, remove and secure-URL methods.

`server/src/adapters/storage/CloudinaryStorageAdapter.js` — Initial Cloudinary implementation.

`server/src/adapters/storage/S3StorageAdapter.js` — Future AWS S3 implementation placeholder.

`server/src/adapters/universities/BaseUniversityAdapter.js` — Defines future application start, document and status methods.

`server/src/adapters/universities/DemoUniversityAdapter.js` — Safe placeholder indicating manual admin processing.

`server/src/adapters/payments/BasePaymentAdapter.js` — Defines checkout, verification and webhook methods.

`server/src/adapters/payments/DemoPaymentAdapter.js` — Development-only simulated payment flow.

`server/src/adapters/email/BaseEmailAdapter.js` — Defines future outbound and inbound email operations.

`server/src/adapters/email/DemoEmailAdapter.js` — Placeholder before a real email provider is connected.

11.18 Events, middleware, validators and constants

`server/src/events/eventBus.js` — Internal event emitter that separates application actions from side effects.

`server/src/events/applicationEvents.js` — Application-created, status-changed and action-required event definitions/handlers.

`server/src/middleware/authenticateSupabaseUser.js` — Verifies the Bearer access token and attaches the trusted Supabase user to the request.

`server/src/middleware/authoriseRole.js` — Loads/checks the local application role before protected controllers run.

`server/src/middleware/validateRequest.js` — Runs request-body, parameter and query validation.

`server/src/middleware/uploadDocument.js` — Multer file type, size and count validation.

`server/src/middleware/rateLimiter.js` — Protects authentication, upload and expensive endpoints from excessive requests.

`server/src/middleware/requestLogger.js` — Records method, path, status and duration without sensitive data.

`server/src/middleware/errorHandler.js` — Produces consistent safe error responses.

`server/src/validators/profileValidator.js` — Profile input schema/rules.

`server/src/validators/resultValidator.js` — Academic-result schema/rules.

`server/src/validators/applicationValidator.js` — Application/cart/status input schema/rules.

`server/src/constants/roles.js` — Canonical role strings.

`server/src/constants/applicationStatuses.js` — Canonical status values and later transition rules.

`server/src/constants/notificationTypes.js` — Canonical notification event names.

`server/src/utils/ApiError.js` — Custom HTTP status and application error-code class.

`server/src/utils/asyncHandler.js` — Forwards rejected asynchronous controller promises to the central error handler.

`server/src/utils/response.js` — Consistent success and error response helpers.

11.19 Express entry points and tests

`server/src/app.js` — Creates and configures Express, middleware, routes, 404 handling and error handling; does not listen on a port.

`server/src/server.js` — Validates configuration, checks database availability and starts the HTTP server for Render.

`server/tests/unit/` — Tests isolated services, strategies, factories and calculations.

`server/tests/integration/` — Tests HTTP routes through controllers, services and a test database.

`server/tests/setup/` — Shared environment, mocks and database cleanup for tests.

`server/package.json` — Optional server-specific dependencies/scripts if the repository uses separate packages.

11.20 Database and documentation files

`database/schema.sql` — Reference schema; begins with comments and later reflects the complete database.

`database/seed.sql` — Reference demonstration data; begins with comments only.

`database/migrations/` — Chronological SQL files that create or alter production tables safely.

`database/seeds/` — Organised demonstration/reference data scripts.

`docs/architecture.md` — Explains the layers, flows and reasons for separation.

`docs/authentication.md` — Documents Supabase-to-Render token verification and local role lookup.

`docs/database-design.md` — Documents table plans, relationships and the Supabase identifier link.

`docs/api-design.md` — Documents route groups, request/response formats and ownership rules.

`docs/deployment.md` — Documents GitHub, Vercel, Render, Supabase and Cloudinary setup.

`docs/storage-strategy.md` — Documents Cloudinary first and S3 later through the adapter interface.

`docs/development-guide.md` — Documents branches, file placement, migrations, testing and naming.

`docs/project-roadmap.md` — Records the agreed implementation order.

12. Security, Privacy and Operational Controls

- Use HTTPS in production.
- Never expose the Supabase service-role key, Cloudinary API secret or DATABASE_URL to the browser.
- Verify every protected request on the backend using the Supabase access token.
- Resolve roles from trusted Render PostgreSQL records rather than browser input.
- Use parameterized PostgreSQL queries.
- Validate all backend input and uploaded files.
- Use role-based access control and ownership checks for student data and documents.
- Use Helmet, correct CORS configuration and rate limiting.

- Avoid logging tokens, identification numbers, passwords or document contents.
- Store document provider identifiers and metadata, not raw database blobs.
- Use secure or signed file delivery rather than public document URLs.
- Record administrator access and status changes in audit logs.
- Plan retention, deletion, consent and access-review rules before production launch.
- Require stronger authentication such as MFA for administrators in production.

Important: The platform will process identity documents, addresses, academic results and possibly information belonging to minors. POPIA compliance, consent, retention and third-party processing require formal legal and security review before a public launch.

13. Development Roadmap

Phase	Outcome
Phase 0	Research, legal review, university/application-channel mapping and source verification.
Phase 1	Project foundation: repository structure, configuration, health endpoint and documentation.
Phase 2	Supabase authentication and local user synchronisation.
Phase 3	Student onboarding and profile sections.
Phase 4	Academic results and profile-completion rules.
Phase 5	Cloudinary document uploads through the storage adapter.
Phase 6	University, programme and admission-rule catalogue.
Phase 7	APS strategies, eligibility and rule-based recommendations.
Phase 8	Persistent cart and requirements validation.
Phase 9	Quotations and demonstration/real provider payment integration.
Phase 10	Application records, state machine, history and student tracking.
Phase 11	Notifications and admin dashboard/processing workflow.
Phase 12	Security hardening, automated tests, audit reporting and production readiness.
Phase 13	Future email aliases, inbound email processing and university automation pilots.
Phase 14	AWS S3 migration and larger-scale background processing if required.

13.1 Recommended first feature after the foundation

Implement Supabase authentication and user synchronisation. A verified Supabase identity should create or retrieve a matching local users record using supabase_user_id. This establishes the security and ownership model needed by every later feature.

14. Codex Starter-Foundation Specification

Codex should receive a self-contained task that creates the repository foundation only. The following requirements capture the agreed prompt without requiring the previous conversation.

14.1 What Codex must create

- Every directory and file described in Sections 10 and 11.
- Valid JavaScript placeholders with comments and minimal exports.
- Semantic HTML boilerplates for all public, student and admin pages.
- CSS starter files with design variables and minimal shared styling.
- A basic Express application using CommonJS consistently.
- GET /api/health returning a successful JSON response.
- Central 404 and error handling.
- A reusable PostgreSQL Pool configuration using DATABASE_URL.
- Supabase browser and server configuration placeholders.
- Bearer-token authentication middleware structure.
- Cloudinary, S3 and storage-factory placeholders.

- Vercel and Render deployment configuration.
- README, architecture, authentication, database, API, storage, deployment and roadmap documentation.
- AGENTS.md coding rules.
- Jest/Supertest starter configuration and optional GitHub CI.

14.2 What Codex must not build yet

- Complete registration or login interfaces.
- Full student profile, result or document functionality.
- The production PostgreSQL schema.
- University or programme data collection.
- APS calculations or recommendations.
- Cart, checkout or payment processing.
- Application workflows, notifications or admin processing.
- Real university automation, scraping or CAPTCHA bypassing.
- Real temporary email accounts or inbound email extraction.

14.3 Starter dependencies

Runtime	Development
express, dotenv, cors, helmet, pg, @supabase/supabase-js, cloudinary, multer, express-rate-limit	nodemon, jest, supertest

14.4 Environment template

```
NODE_ENV=development
PORT=3000
CLIENT_URL=http://localhost:5500
DATABASE_URL=postgresql://username:password@hostname:5432/database_name
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your_public_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_server_only_service_role_key
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
STORAGE_PROVIDER=cloudinary
AWS_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_S3_BUCKET=
```

14.5 Coding rules for generated files

- Do not leave JavaScript files empty.
- Each starter file must explain its future responsibility.
- Use one consistent backend module system: CommonJS.
- Use browser ES modules on the frontend.
- Do not place inline CSS or inline JavaScript in HTML.
- Do not place SQL in routes, controllers or services.
- Do not expose server secrets in client configuration.
- Do not use React, Vue, Angular, TypeScript, Tailwind, Bootstrap or jQuery.
- Do not implement business features beyond safe 501 placeholders.

15. Foundation Acceptance Checklist

☐ All required folders and starter files exist.

☐ The documented npm commands match the generated project structure.
☐ Dependencies install without errors.
☐ The Express server starts successfully.
☐ GET /api/health returns { success: true, message: "UniApply SA API is running." }.
☐ The landing page can be served locally.
☐ JavaScript files contain valid syntax and imports/exports.
☐ The PostgreSQL configuration reads DATABASE_URL safely.
☐ Missing required configuration produces useful messages rather than exposing secrets.
☐ CORS is prepared for the local and future Vercel frontend URL.
☐ Supabase service-role and Cloudinary secret values are absent from frontend files.
☐ Vercel configuration targets the static client only.
☐ Render configuration targets the Express backend and /api/health.
☐ README and project documentation explain setup and current limitations.
☐ No complete MVP feature has accidentally been implemented.

16. Future Extensions and Deferred Work

- Verified data for all South African public universities and relevant application channels.
- Academic-year versioning and scheduled data reviews.
- Real payment-provider adapter.
- Application-specific inbound email aliases and email parsing.
- SMS or WhatsApp notifications.
- Playwright university adapters where explicitly permitted.
- Background queues for email, notifications and automation.
- Historical acceptance modelling only after obtaining reliable and lawful data.
- Advanced career, salary and employment-outlook datasets with dates and methodologies.
- Migration from Cloudinary to AWS S3 by changing STORAGE_PROVIDER and implementing the S3 adapter.
- Mobile applications only after the web workflow is stable.

16.1 Final system principle

Guiding principle: The project should first prove a secure manual workflow: one student profile, explained eligibility, clear missing-information checks, transparent fees, admin processing and complete tracking. Automation is an enhancement after the core workflow is stable and compliant.

Conclusion

UniApply SA is designed as a scalable but practical web platform. The architecture allows the team to work mainly with HTML, CSS and JavaScript while still separating presentation, authentication, business logic, persistence and external providers. The initial Codex task establishes this structure only. The team can then implement authentication, profiles, documents, university data, APS, recommendations, carts, payments, applications and administration in controlled phases.